

DIGITAL NOTES

ON

WEB TECHNOLOGIES

III B.TECH – I SEM



● **DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING**
Sreyas Institute of Engineering and Technology
An Autonomous Institution

Approved by AICTE, Affiliated to JNTUH, Accredited by NAAC-A Grade, NBA (CSE, ECE & ME) & ISO 9001:2015 Certified
Bandlaguda, Nagole, Hyderabad, Telangana 500068

Unit 1

Introduction to PHP: Declaring variables, data types, arrays, strings, operators, expressions, control structures, functions, Reading data from web form controls like text boxes, radio buttons, lists etc., Handling File Uploads. Connecting to database (MySQL as reference), executing simple queries, handling results, Handling sessions and cookies.

File Handling in PHP: File operations like opening, closing, reading, writing, appending, deleting etc. on text and binary files, listing directories.

INTRODUCTION TO PHP

- PHP is an open-source general purpose scripting language, widely used for [website development](#) It is developed by Rasmus Lerdorf in 1994. PHP is a recursive acronym for 'PHP: Hypertext Preprocessor'.
- PHP files can contain text, HTML, JavaScript code, and PHP code. PHP code is executed on the server, and the result is returned to the browser as plain HTML.
- PHP files have a default file extension of ".php"
- PHP is a cross-platform language, capable of running on all major operating system platforms and with most of the web server programs such as Apache, IIS, and nginx.

How PHP works

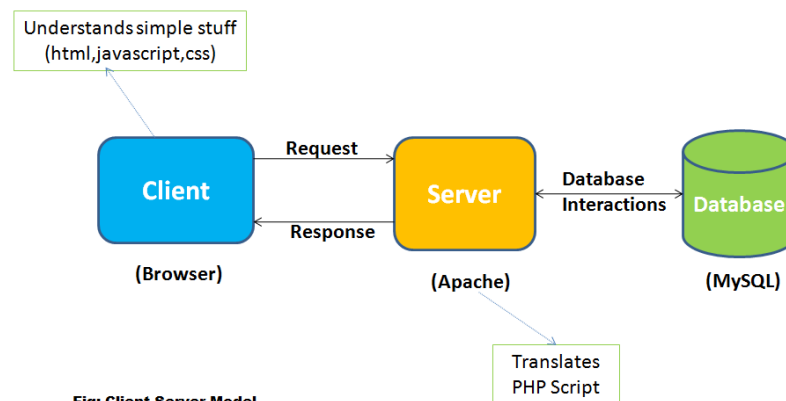


Fig: Client-Server Model

What Can PHP Do

- PHP can generate dynamic page content
- PHP can create, open, read, write, and close files on the server
- PHP can collect form data
- PHP can send and receive cookies
- PHP can add, delete, modify data in your database
- PHP can restrict users to access some pages on your website
- PHP can encrypt data

PHP Installation

To install PHP, We need to install AMP (Apache, MySQL, and PHP) software pack. It is available for all operating systems and it is an open source.

To work with PHP, we must have a web server (Apache), database (MySQL) and PHP scripting language.

AMP (Apache, MySQL, and PHP) software pack for different operating systems:

- WAMP for Windows
- LAMP for Linux
- MAMP for Mac
- SAMP for Solaris
- XAMPP for Cross Platform

To install XAMPP on windows

1. Go to official website of XAMP i.e. <https://www.apachefriends.org/index.html>
2. Click on "**XAMPP for windows**" as per your windows configuration to download XAMPP pack.
3. Double click the **downloaded file** and then click yes to do changes in your operating system.
4. Select the **required components** like MySQL, FileZilla ftp server, PHP, phpMyAdmin or leave the default options and click the Next button.
5. Choose **installation directory** by default installatin directory is "**C:/xampp**".
6. Click the **Allow** access button to allow the XAMPP modules from the Windows firewall.
7. After the installation process, click the **Finish** button of the XAMPP Setup wizard.

PHP Program

To create PHP program, we need basic knowledge of Hyper Text Markup Language (HTML) to generate static content.To create and execute a PHP Program,

Creation:

- Open any text editor (like notepad, edit plus ...).
- Write PHP script and Html tags in that text editor.
- Save this file with "**.php**" extension in the following location.

In Windows:

C: /xampp/htdocs

In Linux (Ubuntu):

/var/www/html

Execution:

- Open the XAMPP control panel and then start the Apache server.
- Open any web browser (Chrome, Firefox ,IE..) and type "**localhost/filename.php**" as url in the address bar.
- Then we get an output of PHP programs.

Basic PHP Code:

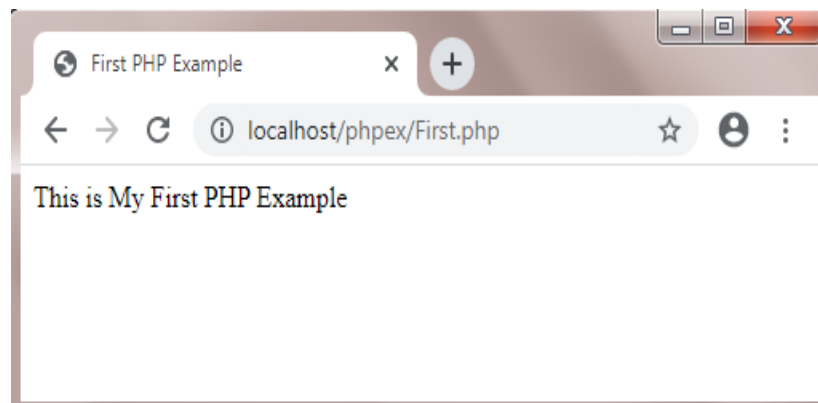
A PHP script can be placed anywhere in the document. A PHP script starts with `<?php` and ends with `?>`:

```
<?php
// PHP code goes here
?>
```

The default file extension for PHP files is ".php".

Example: "First.php"

```
<html>
<head>
<title>First PHP Example</title>
</head>
<body>
<?php
    echo "This is My First PHP Example";
?>
</body>
</html>
```

Output**Comments in PHP**

In general, Comments are used in a programming language to describe the program or to hide the some part of code from the interpreter. Comments in PHP can be used to explain any program code. It can also be used to hide the code as well.

Comment is not a part of the program, but it enhances the interactivity of the program and makes the program readable.

PHP supports two types of comments:

- Single Line Comment
- Multi Line Comment

1. Single Line Comment:

In case the developer wants to specify a single line comment, then the comment must start with " // "

format:

```
// This is single line comment
```

2. Multi Line Comment

In case of Multi lined comment, the comment must starts with /* and ends with */ i.e. /* */ .

format:

```
/* This
```

```
Is
```

```
Multiline comment */
```

Example: "commentdemo.php"

```
<html>
```

```
<head>
```

```
<title>Comments in PHP</title>
```

```
</head>
```

```
<body>
```

```
<h1>Comments in PHP</h1>
```

```
<?php
```

```
// echo prints something on screen
```

```
echo "This is My First PHP Example <br/>";
```

```
/* Comments are used to
```

```
hide or describe the
```

```
code in php */
```

```
echo ' This is an example to demonstrate comments in PHP ';
```

```
?>
```

```
</body>
```

```
</html>
```

Output :**DECLARING VARIABLES**

A variable is a named memory location in which we can store values for the particular program. In other words, Variable is a name of memory location and used to hold the value. Variable can have short names (like x and y) or more descriptive names (age, carname, totalvolume).

Rules for PHP variables:

- A variable starts with the \$ sign, followed by the name of the variable
- A variable name must begin with a letter or the underscore character.
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- A variable name should not contain spaces
- Variable names are case sensitive (\$y and \$Y are two different variables)

Creating Variables

In PHP, a variable is created using **\$ symbol** followed by variable name.

format:

`$variablename = value;`

Declaring variable:

- Variable names can be a group of both letters and digits, but they have to begin with a letter or an underscore, i.e. the variable name can't be starts with numbers or special symbols.

Example:1

- `$a = "Study";` --> Valid declaration
- `$_b = "Glance";` --> Valid declaration
- `$3c = "Hello";` --> Invalid declaration
- `*$d = 25;` ---> Invalid declaration

Example:2

`$a = "Study";`

`$A = "Glance"`

where both variables (\$a and \$A) are different.

PHP is a Loosely Typed Language

In PHP, We don't need to declare explicitly variable, when we assign any value to the variable that variable is declared automatically. In PHP, We don't need to specify the type of variable because PHP is a loosely typed language. I.e. In loosely typed language no need to specify the type of variable because the variable automatically changes it's datatype based on the assigned value.

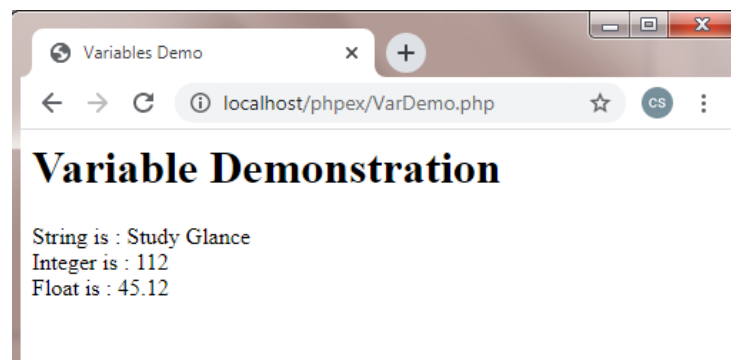
Example:

```
$str = "Study Glance";    //string type variable
$x = 250;                 //integer type variable
$y = 14.6;                //float type variable
```

Example: "VarDemo.php"

```
<html>
<head>
<title>Variables Demo</title>
</head>
<body bgcolor="#fff">
<h1>Variable Demonstration</h1>
<?php
$str = "Study Glance";
$x = 112;
$y = 45.12;
echo "String is : $str<br/>";
echo "Integer is : $x <br/>";
echo "Float is : $y <br/>";
?>
</body>
</html>
```

Output :



Variable Scopes

The scope of a variable is the part of the script where the variable can be referenced/used. PHP has four different variable scopes:

- local
- global
- static

Local Scope

A variable declared within a PHP function is local and can only be accessed within that function:

Example

```
<?php
$x=5; // global scope
function myTest()
echo $x; // local scope
myTest();
```

Global Scope

A variable that is defined outside of any function, has a global scope. Global variables can be accessed from any part of the script, EXCEPT from within a function. To access a global variable from within a function, use the global keyword:

Example

```
<?php
$x=5; // global scope
$y=10; // global scope
function myTest()
global $x,$y;
$y=$x+$y;
myTest();
echo $y;
?>
```

Output:

15

Static Scope

When a function is completed, all of its variables are normally deleted. However, sometimes you want a local variable to not be deleted. › To do this, use the static keyword when you first declare the variable:

Example

```
<?php
function myTest()
```

```
static $x=0; echo $x;
$ ++;
myTest();
myTest();
myTest();
?>
```

PHP ECHO AND PRINT STATEMENTS

In PHP there is two basic ways to get output: echo and print. There are some differences between echo and print

echo - can output one or more strings

print - can only output one string, and returns always 1

Tip: echo is marginally faster compared to print as echo does not return any value.

```
<?php
```

```
echo "<h2>PHP is fun!</h2>"; echo "Hello world!<br>";
```

```
echo "This", " string", "was", " made", " with multiple
parameters.";
```

DATATYPES IN PHP

The datatypes are used to hold different types of data (or) values. Datatypes specifies the different types of data that are supported in PHP.

PHP supports total eight primitive datatypes :

- Integer
- Float
- String
- Boolean
- Array
- Object
- NULL
- Resource

Note :In PHP, var_dump() function returns the data type and value. Ex: int(123).

Integer :

In PHP, Integers hold whole numbers including positive and negative numbers (... -3, -2, -1, 0, 1, 2, 3, ...). i.e., numbers without decimal point. They can be decimal (base 10), octal (0) or hexadecimal (0x) optionally preceded by + or - sign.

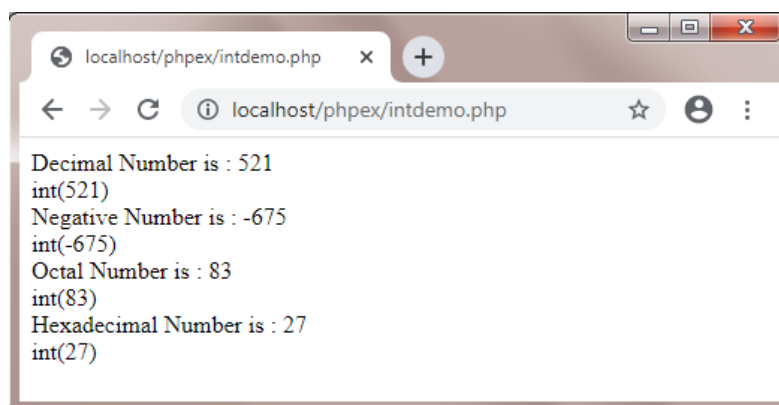
Example: "intdemo.php"

```
<?php
$var1 = 521; // decimal number
echo "Decimal Number is : ".$var1."<br>";
var_dump($var1); // it prints datatype and value
echo "<br>";

$var2 = -675; // a negative number
echo "Negative Number is : ".$var2."<br>";
var_dump($var2);
echo "<br>";

$var3 = 0123; // octal number
echo "Octal Number is : ".$var3."<br>";
var_dump($var3);
echo "<br>";

$var4 = 0x1B; // hexadecimal number
echo "Hexadecimal Number is : ".$var4."<br>";
var_dump($var4);
?>
```

Output:

Float :

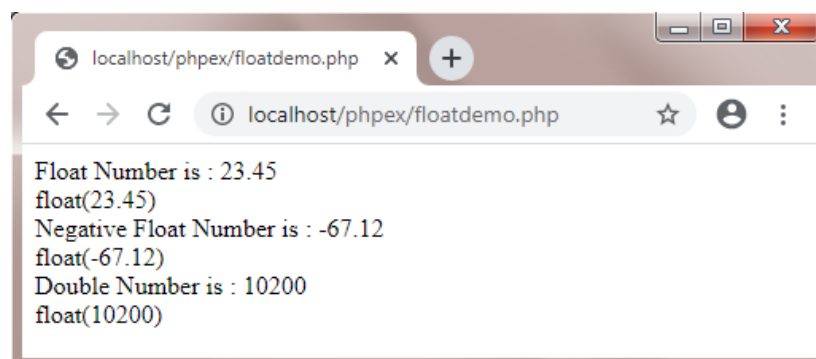
A float (floating point number) is a number with a decimal point. It hold numbers containing fractional or decimal part including positive and negative numbers.

Example: "floatdemo.php"

```
<?php
$var1 = 23.45; // float
echo "Float Number is : ".$var1."<br>";
var_dump($var1); // it prints datatype and value
echo "<br>";
```

```
$var2 = -67.12; // a negative float
echo "Negative Float Number is : ".$var2."<br>";
var_dump($var2);
echo "<br>";
```

```
$var3 = 10.2e3; // double
echo "Double Number is : ".$var3."<br>";
var_dump($var3);
?>
```

Output:**String :**

A string is a sequence of characters which are enclosed with single quotes or double quotes.

Example: "stringdemo.php"

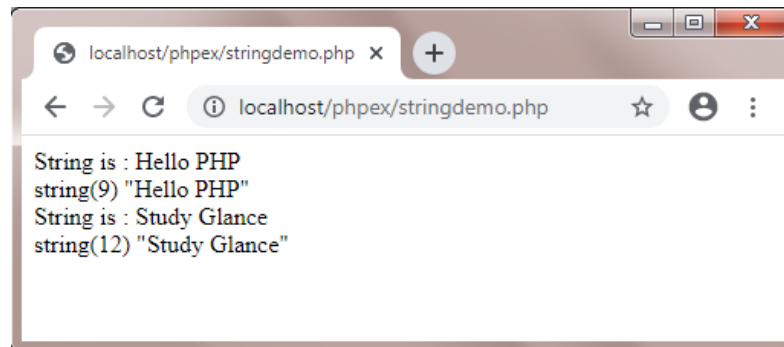
```
<?php
$var1 = "Hello PHP"; // string
echo "String is : ".$var1."<br>";
var_dump($var1); // it prints datatype and value
echo "<br>";
```

```

$var2 = 'Study Glance'; // string
echo "String is : ".$var2."<br>";
var_dump($var2);
echo "<br>";
?>

```

Output :



Boolean :

A boolean datatype hold two possible values either TRUE or FALSE.

Example: "blndemo.php"

```

<?php
$a= TRUE; // boolean
echo "Status is : ".$a."<br>";
var_dump($a); // it prints datatype and value
echo "<br>";

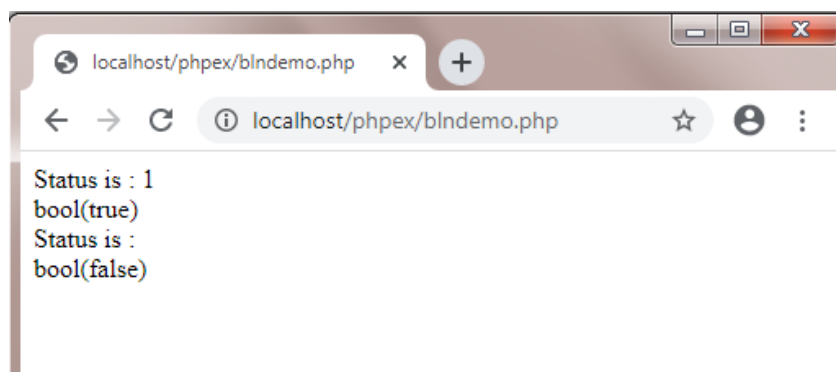
```

```

$b= FALSE; // boolean
echo "Status is : ".$b."<br>";
var_dump($b); // it prints datatype and value
?>

```

Output :

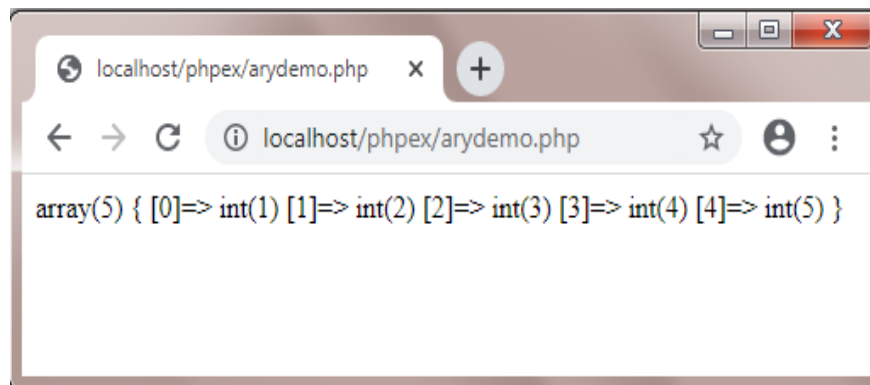


Array :

An array datatype can hold more than one value at a time. In other words, An array is collection of elements or items.

Example: "arydemo.php"

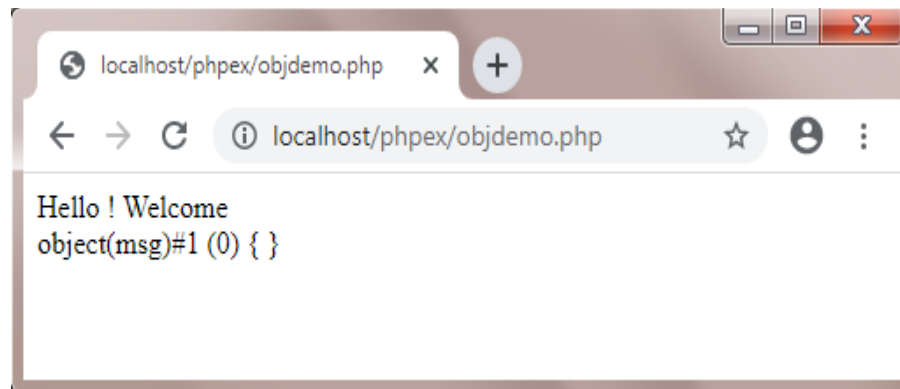
```
<?php
$numbers = array(1,2,3,4,5); //array
var_dump($numbers); // prints datatype and value
echo "<br>";
?>
```

Output :**Object :**

In PHP, Object is an instance of user defined class that can store both values and functions. Objects are created based on user defined class via the new keyword

Example: "objdemo.php"

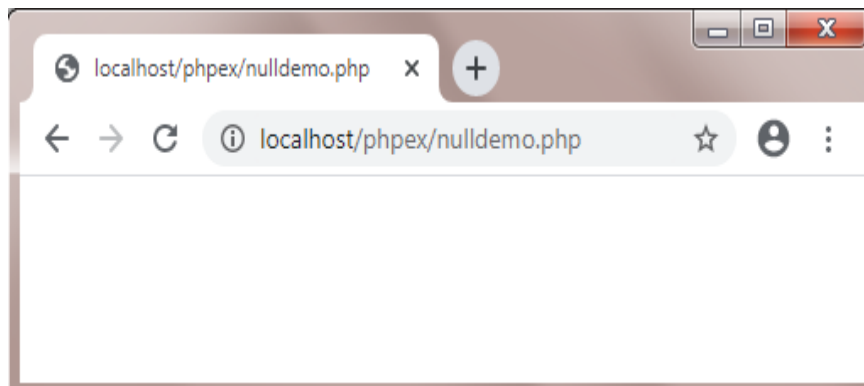
```
<?php
class msg {
    function display() {
        $str = "Hello ! Welcome ";
        echo $str;
    }
}
$obj = new msg();
$obj -> display();
echo "<br>";
var_dump($obj);
?>
```

Output :**Null :**

In PHP, Null is a special data type that has only one value: NULL, And it is case sensitive you must write in capital letters. The special type of data type NULL defined a variable with no value.

Example: "nulldemo.php"

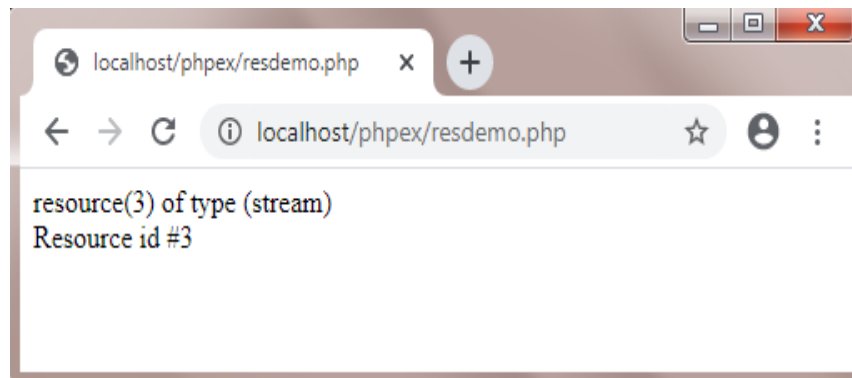
```
<?php
    $var = NULL;
    echo $var; //it will not give any output
?>
```

Output :**Resource :**

In PHP, A resource is a special variable, that hold a reference to an external resource such as database connection and opened files. The Resources are not exact data type in PHP

Example: "resdemo.php"

```
<?php
// Open a file for reading
$fp = fopen("file.txt", "r");
var_dump($fp);
echo "<br>";
echo $fp;
?>
```

Output :**ARRAYS**

An array is a special variable, which can hold more than one value at a time. In other words, an array can hold many values under single name, and we can access the values by referring to an index .

In PHP, the **array()** function is used to create an array.

PHP supports three types of arrays, those are

- **Indexed** array
- **Associative** array
- **Multi-dimensional** array

Indexed array :

In arrays, the elements are assigned to specific index to access those elements easily. In the Indexed array, index is represented by number which starts from '0'.

An Indexed array is array with collection of elements where index is represented by number.

There are two ways to define indexed array, those are

1st-way:

```
$marks = array(60, 72, 66);
```

2nd-way:

```
$marks [0] = 60;
```

```
$marks [1] = 72;
```

```
$marks [2] = 66;
```

Example: "IndArray.php"

```
<html>
```

```
<head>
```

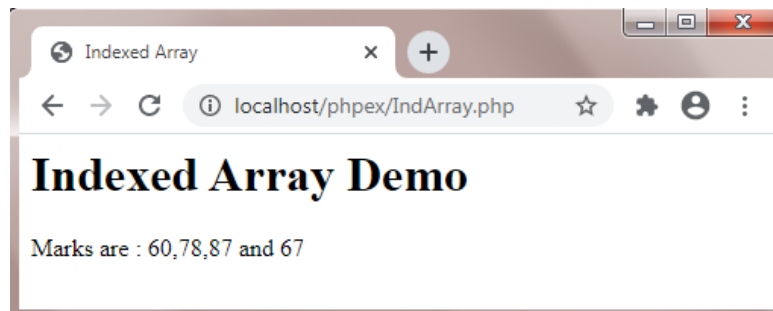
```
<title>Indexed Array</title>
```

```

</head>
<body>
<h1>Indexed Array Demo</h1>
<?php
$marks=array(60,78,87,67);
echo "Marks are : $marks[0],$marks[1],$marks[2] and $marks[3]";
?>
</body>
</html>

```

Output :



Associative array :

The associative arrays are very similar to indexed arrays in terms of functionality but they are different in terms of their index.

Associative array will have their index as string so that we can establish a strong association between key and value. In PHP, we can associate name with each array element using "=>" symbol.

There are two ways to define associative array, those are

1st-way:

```
$marks = array("C"=>60, "Java"=>72, "Php"=>66);
```

2nd-way:

```
$marks ["C"] = 60;
```

```
$marks ["Java"] = 72;
```

```
$marks ["Php"] = 66;
```

Example: "AssocArray.php"

```

<html>
<head>
<title>Associate Array</title>
</head>

```

```

<body>
<h1>Associate Array Demo</h1>
<?php
$marks=array("WT"=>56,"FLAT"=>67,"SE"=>65,"PPL"=>78);
echo "Marks of WT :".$marks["WT"]."<br/>";
echo "Marks of FLAT :".$marks["FLAT"]."<br/>";
echo "Marks of SE :".$marks["SE"]."<br/>";
echo "Marks of PPL :".$marks["PPL"]."<br/>";
?>
</body>
</html>

```

Output :



Multidimensional array :

The multidimensional array is also known as array of arrays. It allows you to store tabular data in an array.

Multidimensional array represents in the form of matrix(i.e. rows and columns).

The following way is to define multidimensional array

```

$employees = array (
    array (1,"A",28),
    array (2,"B",27),
    array (3,"C",29)
);

```

Example: "MultiArray.php"

```

<html>
<head>

```

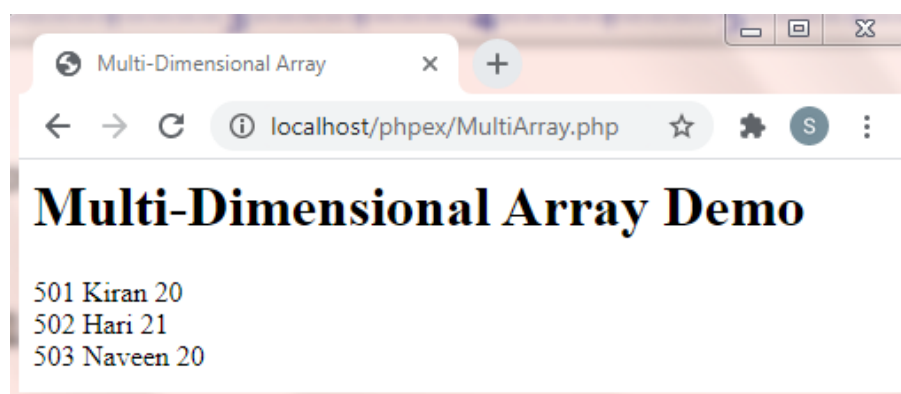
```

<title>Multi-Dimensional Array</title>
</head>
<body>
<h1>Multi-Dimensional Array Demo</h1>
<?php
$students=array(
    array(501,"Kiran",20),
    array(502,"Hari",21),
    array(503,"Naveen",20)
);
for($i=0;$i<3;$i++)
{
    for($j=0;$j<3;$j++)
    {
        echo $students[$i][$j]. " ";
    }
    echo "<br/>";
}

?>
</body>
</html>

```

Output :



STRINGS

In PHP, A String is a sequence of characters which are enclosed with single quotes or double quotes. There are 2 ways to specify string

- Single Quotes **Ex:** \$str=' Hello PHP ';
- Double Quotes **Ex:** \$str=" Hello PHP ";

Where in Single quoted string, we can store multiline text, special characters and escape sequences. Where in Double quoted string, we can't use special characters directly.

Double quoted string :

Example:

```
$age = 20;
$str1="Your age is : $age";           // string1
echo $str1;
$str2="Your age is : '$age'"; // string2
echo $str2;
```

Output:

```
Your age is : 20
Your age is : '20'
```

Single quoted string :

Example:

```
$age = 20;
$str1='Your age is : $age';           //string1
echo $str1;
$str2='Your age is : "$age"'; //string2
echo $str2;
```

Output:

```
Your age is : $age
Your age is : '$age'
```

String Function in PHP

PHP provides various string functions to perform different operations on strings i.e. to access and manipulate strings. Those are,

strtolower() :

It returns string in lowercase letters i.e. it converts given string into lowercase.

Example:

```
$str="StudyGLANCE";
$str1=strtolower($str);
echo $str1;
```

Output:

studyglance

strtoupper() :

It returns string in uppercase letters i.e. it converts given string into uppercase.

Example:

```
$str="StudyGLANCE";  
$str1=strtoupper($str);  
echo $str1;
```

Output:

STUDYGLANCE

ucwords() :

It converts first character of each word in given string into uppercase.

Example:

```
$str="study glance";  
$str1=ucwords($str);  
echo $str1;
```

Output:

Study Glance

strlen() :

It returns the length of given string.

Example:

```
$str="study glance";  
$len=strlen($str);  
echo "Length of given string : $len";
```

Output:

Length of given string : 12

strrev() :

It returns the given string in reverse.

Example:

```
$str="study glance";
```

```
$str1=strrev($str);  
echo $str1;
```

Output:

```
ecnalg yduts
```

str_word_count() :

It returns the number of words in the given string.

Example:

```
$str="study glance";  
$wc=str_word_count($str);  
echo $wc;
```

Output:

```
2
```

strpos() :

It searches for a specific text within a string. If a match is found, this function returns the character position of the first match. If no match is found, it will return false.

Example:

```
$str="study glance";  
$pos=strpos($str,"glance");  
echo $pos;
```

Output:

```
6
```

str_replace() :

It replaces some characters with another characters in the given string.

Example:

```
$str="study glance";  
$str1=str_replace("glance","world",$str);  
echo $str1;
```

Output:

```
study world
```

substr() :

It returns the sub-string from the given string.

Example:

```
$str="study glance";
$str1=substr($str,6);
$str2=substr($str,0,4);
echo $str1;
echo $str2;
```

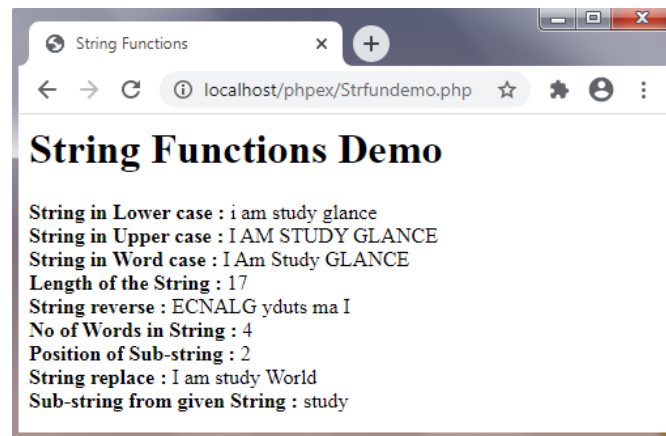
Output:

```
glance
study
```

Example: "Strfundemo.php"

```
<html>
<head>
<title>String Functions</title>
</head>
<body>
<h1>String Functions Demo</h1>
<?php
$str="I am study GLANCE";
echo "<strong>String in Lower case : </strong>".strtolower($str)."<br/>";
echo "<strong>String in Upper case :</strong> ".strtoupper($str)."<br/>";
echo "<strong>String in Word case :</strong> ".ucwords($str)."<br/>";
echo "<strong>Length of the String :</strong> ".strlen($str)."<br/>";
echo "<strong>String reverse :</strong> ".strrev($str)."<br/>";
echo "<strong>No of Words in String :</strong> ".str_word_count($str)."<br/>";
echo "<strong>Position of Sub-string :</strong> ".strpos($str,"am")."<br/>";
echo "<strong>String replace :</strong> ".str_replace("GLANCE","World",$str)."<br/>";
echo "<strong>Sub-string from given String :</strong> ".substr($str,5,5)."<br/>";
?>
</body>
</html>
```

Output :



OPERATORS

An operator can be defined as a symbol which is used to perform a particular operation between operands.

PHP provides a variety of operators described as follows.

- Arithmetic Operators
- Assignment Operators
- Comparison Operators
- Increment/Decrement Operators
- Logical Operators
- String Operators
- Array Operators

Arithmetic operators

Arithmetic operators are used to perform arithmetic operations between two operands.

Operator	Description
+	(addition)
-	(subtraction)
*	(multiplication)
/	(divide)
%	(reminder)
**	(exponent)

Assignment Operators

The assignment operators are used to assign the value of the right expression to the left operand.

Operator	Description
= (Assigns to)	Assigns values from right side operands to left side operand.
+= (Assignment after Addition)	It adds right operand to the left operand and assign the result to left operand.
-= (Assignment after Subtraction)	It subtracts right operand from the left operand and assign the result to left operand.
*= (Assignment after Multiplication)	It multiplies right operand with the left operand and assign the result to left operand.
/= (Assignment after Division)	It divides left operand with the right operand and assign the result to left operand.
%= (Assignment after Modulus)	It takes modulus using two operands and assign the result to left operand.
**= (Assignment after Exponent)	Performs exponential (power) calculation on operators and assign value to the left operand.

Comparison operators

Comparison operators are used to comparing the value of the two operands and returns boolean true or false accordingly.

Operator	Description
== (Equal to)	Equal to - True if both operands are equal.
!= (Not equal to)	Not equal to - True if operands are not equal.
<= (Less than or equal)	Less than or equal to - True if left operand is less than or equal to the right
>= (Greater than or equal)	Greater than or equal to - True if left operand is greater than or equal to the right
< (Less than)	Less than - True if left operand is less than the right
> (Greater than)	Greater than - True if left operand is greater than the right

Operator	Description
=== (Identical)	Identical - True if both operands have same value and datatype.
!== (Not Identical)	Not Identical - True if both operands doesn't have same value and datatype..

Increment/Decrement operators

Increment/Decrement operators are used to increment or decrement the value of the operand.

Operator	Description
++ (Increment)	Increment - It increments the value of operand by one.
-- (Decrement)	Decrement - It decrements the value of operand by one.

Logical Operators

The logical operators are used primarily in the expression evaluation to make a decision.

Operator	Description
and (or) && (logical and)	Returns True if both statements are true
or (or) (logical or)	Returns True if one of the statements is true
not (or) ! (logical not)	Reverse the result, returns False if the result is true

String operators

String operators are used to concatenate two strings.

Operator	Description
. (Concatenation)	It concatenates left operand and right operand.
.= (Assignment after Concatenation)	It concatenates left operand and right operand and assign the result to left operand.

Array operators

Array operators are used to perform various operations on arrays.

Operator	Description
+	It combine two arrays.
== (Equal to)	Equal to - True if both operands(arrays) are equal.
!= (Not equal to)	Not equal to - True if operands(arrays) are not equal.
=== (Identical)	Identical - True if both operands(arrays) have same value and datatype.
!== (Not Identical)	Not Identical - True if both operands(arrays) doesn't have same value and datatype..

Example: "OperatorDemo.php"

```

<html>
<head>
<title>Operators Demo</title>
</head>
<body>
<?php
$a=10;
$b=5;
//Arithmetic operators
$c=$a+$b;
echo "Addition is :$c <br/>";
$c=$a-$b;
echo "Subtraction is :$c <br/>";

//Increment/Decrement Operators
$c=$a++;
echo "Increment is :$c <br/>";
$c=$a--;
echo "Decrement is :$c <br/>";

//Assignment Operators
$c+=$a;
echo " Add Assignment is :$c <br/>";

```

```

$c-=$a;
echo "Sub Assignment is :$c <br/>";

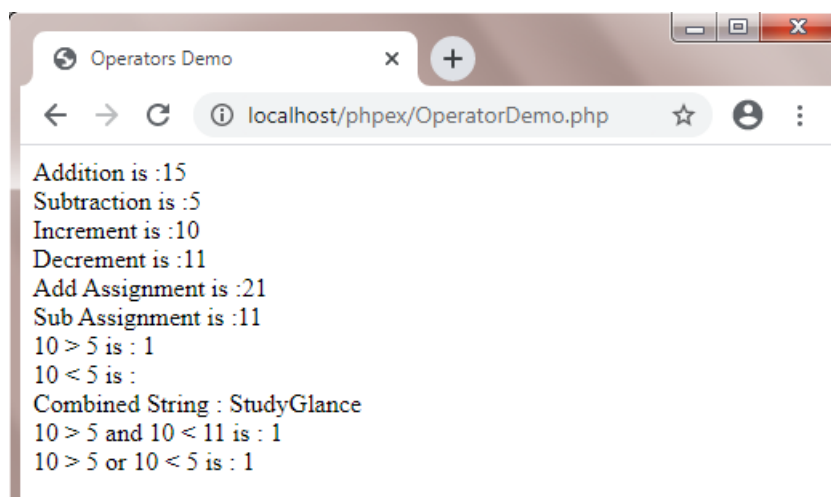
//Comparison operators
echo "$a > $b is : ".$a>$b)."<br/>";
echo "$a < $b is : ".$a<$b)."<br/>";

//String operators
$str1="Study";
$str2="Glance";
$str3=$str1.$str2;
echo "Combined String : $str3 <br/>";

//Logical Operators
echo "$a > $b and $a < $c is : ".$a>$b && $a<$c)."<br/>";
echo "$a > $b or $a < $b is : ".$a>$b || $a<$b)."<br/>";
?>
</body>
</html>

```

Output :



PHP EXPRESSIONS

Almost everything in a PHP script is an expression. Anything that has a value is an expression. In a typical assignment statement ($\$x=100$), a literal value, a function or operands processed by operators is an expression, anything that appears to the right of assignment operator ($=$)

Syntax

`$x=100; //100 is an expression`

`$a=$b+$c; //b+$c is an expression`

`$c=add($a,$b); //add($a,$b) is an expression`

`$val=sqrt(100); //sqrt(100) is an expression`

`$var=$x!=$y; //$x!=$y is an expression`

CONTROL STRUCTURES IN PHP

The Control Structures (or) Statements used to alter the execution process of a program.

These statements are mainly categorized into following:

- Conditional Statements
- Loop Statements
- Jump Statements

Conditional Statements :

Conditional Statements performs different computations or actions depending on conditions. In PHP, the following are conditional statements

- **if** statement
- **if - else** statement
- **if - elseif - else** statement
- **switch** statement

if statement

The if statement is used to test a specific condition. If the condition is true, a block of code (if-block) will be executed.

Syntax :

```
if (condition)
{
    statements
}
```

if - else statement

The if-else statement provides an else block combined with the if statement which is executed in the false case of the condition.

Syntax :

```
if (condition)
{
    statements
}
```

```

}
else
{
    statements
}

```

if - elseif - else statement

The elseif statement enables us to check multiple conditions and execute the specific block of statements depending upon the true condition among them.

Syntax :

```

if (condition1)
{
    statements
}
else if (condition2)
{
    statements
}
else if (condition3)
{
    statements
}
else
{
    statements
}

```

Example: "ConditionalDemo.php"

```

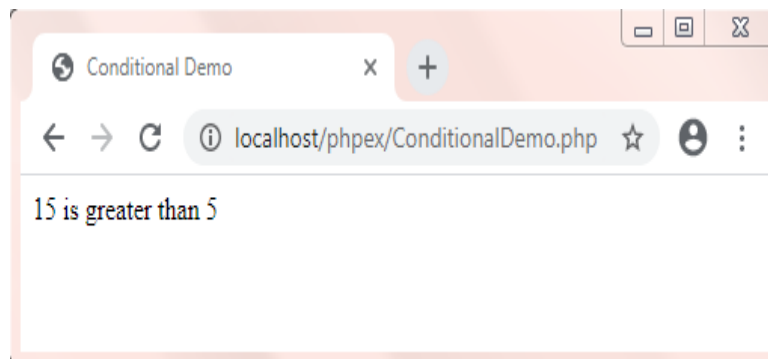
<html>
<head>
<title>Conditional Demo</title>
</head>
<body>
<?php
$x=15;
$y=5;
if ($x > $y)
{

```

```

    echo "$x is greater than $y";
}
else if ($x < $y)
{
    echo "$x is less than $y";
}
else
{
    echo "Both are Equal";
}
?>
</body>
</html>

```

Output :**switch statement**

The switch statement enables us to execute a block of code from multiple conditions depending upon the expression.

Syntax :

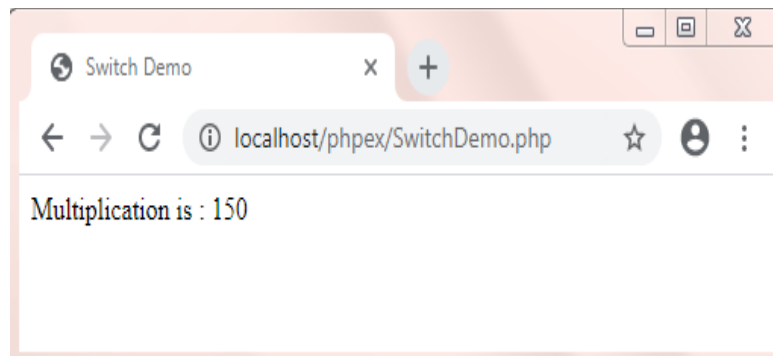
```

switch (expression)
{
case 1: statements
    break;
case 2: statements
    break;
case 3: statements
    break;
default: statements
}

```

Example: "SwitchDemo.php"

```
<html>
<head>
<title>Switch Demo</title>
</head>
<body>
<?php
    $x=15;
    $y=10;
    $op='*';
    switch($op)
    {
        case '+':  $z = $x + $y;
                    echo "Addition is : $z";
                    break;
        case '-':  $z = $x - $y;
                    echo "Subtraction is : $z";
                    break;
        case '*':  $z = $x * $y;
                    echo "Multiplication is : $z";
                    break;
        case '/':  $z = $x / $y;
                    echo "Division is : $z";
                    break;
        case '%':  $z = $x % $y;
                    echo "Modulus is : $z";
                    break;
        default:   echo "Invalid Operator";
    }
?>
</body>
</html>
```

Output :**Loop Statements :**

Sometimes we may need to alter the flow of the program. If the execution of a specific code may need to be repeated several numbers of times then we can go for loop statements.

In PHP, the following are loop statements

- **while** loop
- **do - while** loop
- **for** loop

while loop statement

With the while loop we can execute a set of statements as long as a condition is true. The while loop is mostly used in the case where the number of iterations is not known in advance.

Syntax :

```
while (condition)
```

```
{
    statements
}
```

Example: "WhileDemo.php"

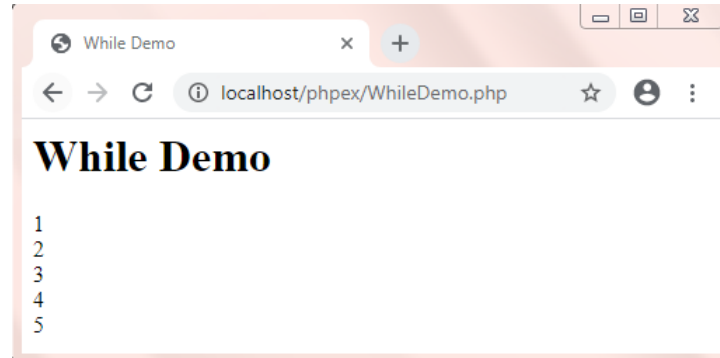
```
<html>
<head>
<title>While Demo</title>
</head>
<body>
<h1>While Demo</h1>
<?php
$n=1;
while($n<=5)
{
    echo "$n <br/>";
```

```

    $n++;
}
?>
</body>
</html>

```

Output :



do - while loop statement

The do-while loop will always execute a set of statements atleast once and then execute a set of statements as long as a condition is true.

Syntax :

```

do
{
    statements
} while (condition);

```

Example: "DoWhileDemo.php"

```

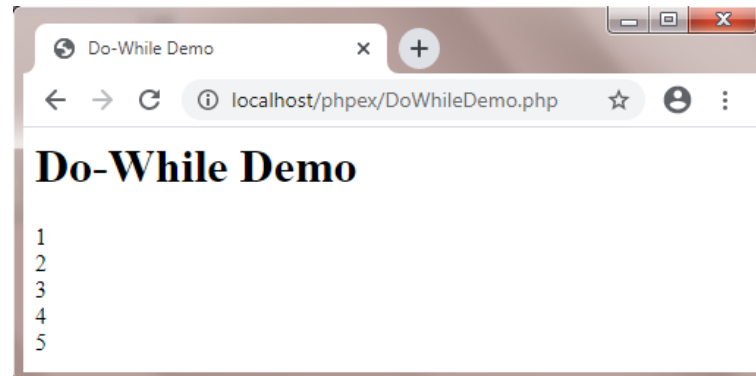
<html>
<head>
<title>Do-While Demo</title>
</head>
<body>
<h1>Do-While Demo</h1>
<?php
$n=1;
do
{
    echo "$n <br/>";
    $n++;
} while($n<=5);
?>

```

```
</body>
```

```
</html>
```

Output :



for loop statement

With the for loop we can execute a set of statements specified number of times. The for loop is mostly used in the case where the number of iterations is known in advance..

Syntax :

```
for (initialization; condition; increment/decrement)
```

```
{
    statements
}
```

Example: "ForDemo.php"

```
<html>
```

```
<head>
```

```
<title>For Demo</title>
```

```
</head>
```

```
<body>
```

```
<h1>For Demo</h1>
```

```
<?php
```

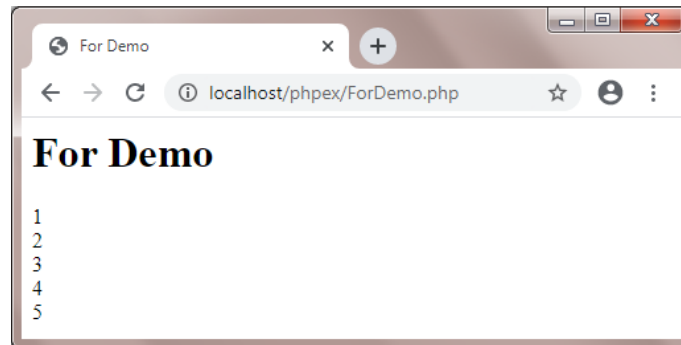
```
for($i=1;$i<=5;$i++)
```

```
{
    echo "$i <br/>";
}
```

```
?>
```

```
</body>
```

```
</html>
```

Output :**Jump Statements :**

Jump statements in PHP are used to alter the flow of a loop like you want to skip a part of a loop or terminate a loop.

In PHP, the following are jump statements

- **break** statement
- **continue** statement

break statement

The break is a keyword in php which is used to bring the program control out of the loop. i.e. when a break statement is encountered inside a loop, the loop is terminated and program control resumes at the next statement following the loop.

The break statement breaks the loops one by one, i.e., in the case of nested loops, it breaks the inner loop first and then proceeds to outer loops. The break is commonly used in the cases where we need to break the loop for a given condition.

Syntax :

```
break;
```

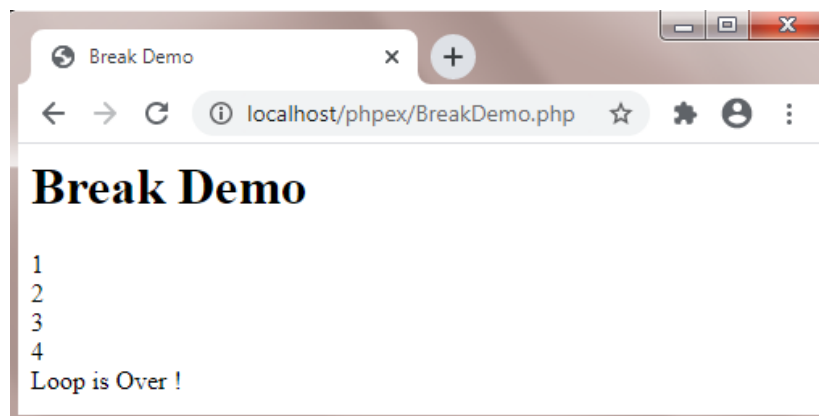
Example: "BreakDemo.php"

```
<html>
<head>
<title>Break Demo</title>
</head>
<body>
<h1>Break Demo</h1>
<?php
for($i=1;$i<=10;$i++)
{
    if($i==5)
```

```

{
    break;    //terminates the current loop
}
echo "$i <br/>";
}
echo "Loop is Over !";
?>
</body>
</html>

```

Output :**continue statement**

The continue statement in php is used to bring the program control to the beginning of the loop. i.e. when a continue statement is encountered inside the loop, remaining statements are skipped and loop proceeds with the next iteration.

The continue statement skips the remaining lines of code inside the loop and start with the next iteration. It is mainly used for a particular condition inside the loop so that we can skip some specific code for a particular condition.

Syntax :

```
continue;
```

Example: "ContinueDemo.php"

```

<html>
<head>
<title>Continue Demo</title>
</head>
<body>
<h1>Continue Demo</h1>
<?php

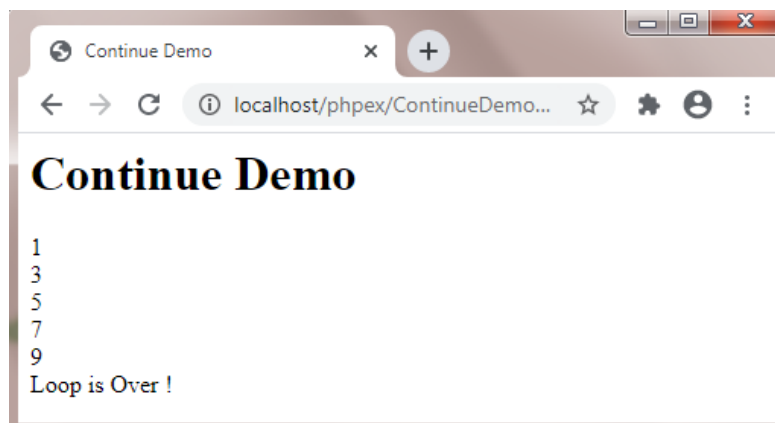
```

```

for($i=1;$i<=10;$i++)
{
    if($i%2==0)
    {
        continue; //terminates the current Iteration and moves to Next
    }
    echo "$i <br/>";
}
echo "Loop is Over !";
?>
</body>
</html>

```

Output :



FUNCTIONS

A function is a piece of code or self contained block of code that is used to perform a particular task. The main advantage of functions is that code reusability (Write once Invoke multiple).

PHP supports both built-in and user-defined functions. And PHP supports thousands of built-in functions and allows us to define own functions, by using "**function**" keyword.

Example: "FunSimple.php"

```

<html>
<head>
<title>Function Example</title>
</head>
<body>
<h1>Simple Function Example</h1>
<?php
function simple() //Function Definition
{
    echo "Welcome to php";
}

```

```

}
simple();//Function calling
?>
</body>
</html>

```

Types of Functions in PHP

[PHP](#) has two types of functions:

1. User-Defined Functions

A user-defined function is created to perform a specific task as per the developer's need. These functions can accept parameters, perform computations, and return results.

```

<?php
function addNumbers($a, $b)
{ return $a + $b;
}
echo addNumbers(5, 3);
?>

```

2. Built-in Functions

PHP comes with many built-in functions that can be directly used in your code. For example, `strlen()`, `substr()`, `array_merge()`, `date()` and etc are built-in PHP functions. These functions provide useful functionalities, such as string manipulation, date handling, and array operations, without the need to write complex logic from scratch.

```

<?php
$str = "Hello, World!";
echo strlen($str); // Output: 13
?>

```

PHP Function Parameters and Arguments:

Functions can accept input values, known as parameters or arguments.

- Parameters are variables defined in the function declaration that accept values. Arguments are the actual values passed to the function when it is called. These values can be passed to the function in two ways:

Passing by Value : When you pass a value by reference, the function works with a copy of the argument. This means the original value remains unchanged.

```

<?php
function add($x, $y) {
    $x = $x + $y;
    return $x;
}

```

```
echo add(2, 3); // Outputs: 5
```

```
?>
```

Passing by Reference

By passing an argument by reference, any changes made inside the function will affect the original variable outside the function.

```
<?php
```

```
    function addByRef(&$x, $y)
```

```
    {
```

```
        $x = $x + $y;
```

```
    }
```

```
    $a = 5;
```

```
    addByRef($a, 3);
```

```
    echo $a;
```

```
?>
```

Default argument function

PHP allow us to define default argument values in the function. In this case if we don't pass any value to the function, it will use default argument value.

Example: "FunDefaultParamd.php"

```
<html>
```

```
<head>
```

```
<title>Default Argument Function</title>
```

```
</head>
```

```
<body>
```

```
<h1>Default Argument Function Demo</h1>
```

```
<?php
```

```
function add($a=10, $b=10) //Function Definition
```

```
{
```

```
    $sum = $a + $b;
```

```
    echo "Addition is : $sum <br/>";
```

```
}
```

```
add(); //Function calling with no parameter
```

```
add(20); //Function calling with 1 parameter
```

```
add(20,40); //Function calling with 2 parameter
```

```
?>
```

```
</body>
```

```
</html>
```

Output:**Recursive function**

PHP allow us to use recursive function. In this case, we call the current function within the same function. It is also known as recursion.

```
<?php
function factorial($n) {
    if ($n == 0) {
        return 1;
    } else {
        return $n * factorial($n - 1);
    }
}
echo factorial(5);
?>
```

READING DATA FROM FORMS

In PHP, We can create and use forms to get data from a Form. To read or get data from a form, we need to use SuperGlobal Variables. A Superglobal variable is a built in PHP variable that is available in any scope.

PHP supports two superglobal variables, Those are

- \$_GET
- contains list of all input element names and values sent by a form using the get method.
- \$_POST
- contains list of all input element names and values sent by a form using the post method.

In simple words, the form request generated through may be get or post method. To retrieve data from get request, we need to use \$_GET, To retrieve data from post request, we need to use \$_POST.

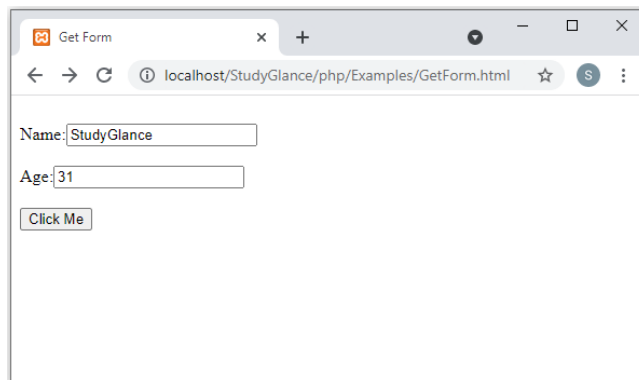
Get Form :

Get form is a form which is generated through get request. The get request is the default form request. The data passed through get request is visible on the URL so it is not secured. And we can able send limited amount of data through get request.

Here, we see an example to work with get request form, which consists two programs one is HTML file(GetForm.html) to create an user interface and second one is PHP file(getform.php) to read data from HTML input elements.

Example:**GetForm.html**

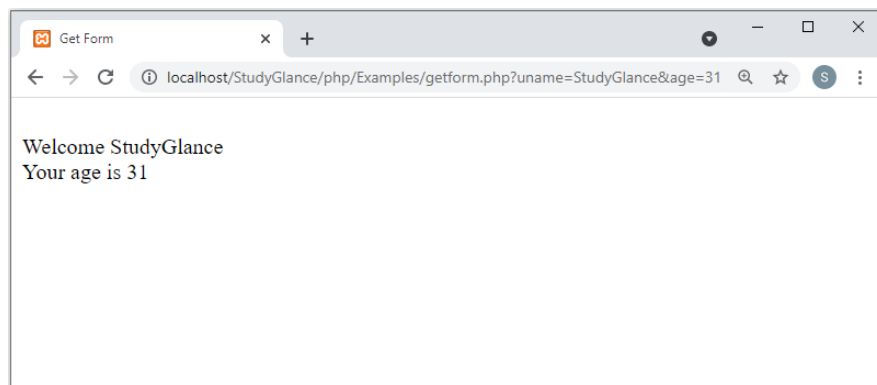
```
<!doctype html>
<html lang="en">
  <title>Get Form</title>
</head>
<body><br>
  <form action="getform.php" method="get">
    Name:<input type="text" name="uname"></input><br/><br/>
    Age:<input type="number" name="age"></input><br/><br/>
    <input type="submit" value="Click Me"></input>
  </form>
</body>
</html>
```

Output:

When the user clicks on the "Click Me" button the following PHP script file gets executed. The PHP script reads the data from HTML form elements using \$_GET superglobal variable and then displays on the screen.

getform.php

```
<!doctype html>
<html lang="en">
  <title>Get Form</title>
</head>
<body><br>
<?php
  // To read data from input elements using $_GET
  $name = $_GET["uname"];
  $age = $_GET["age"];
  // To display data
  echo "Welcome ".$name."<br/>";
  echo "Your age is ".$age;
?>
</body>
</html>
```

Output:**Post Form :**

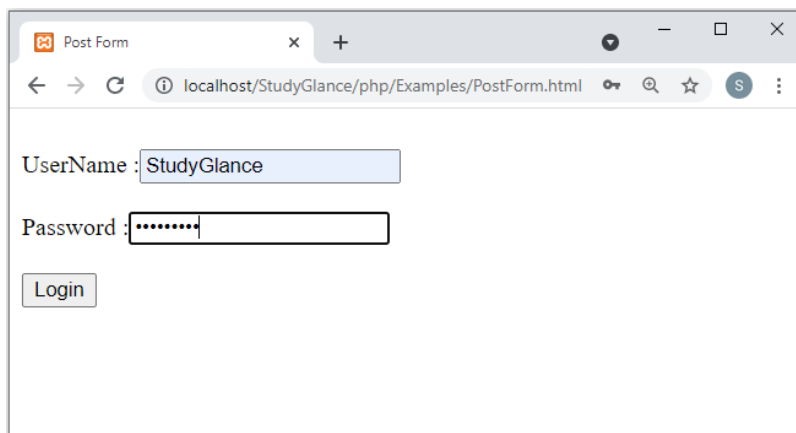
Post form is a form which is generated through post request. The post request is not a default form request. The data passed through post request is not visible on the URL so it is secured. And it allows you send unlimited amount of data.

The Post request is widely used to submit form that have large amount of data such as file upload and sensitive data such as login form, registration form etc.

Here, I am attaching an example to work with post request form, which consists two programs one is HTML file(PostForm.html) to create an user interface and second one is PHP file(postform.php) to read data from HTML input elements.

Example:**PostForm.html**

```
<!doctype html>
<html lang="en">
  <title>Post Form</title>
</head>
<body><br>
  <form action="postform.php" method="post">
    UserName :<input type="text" name="uname"></input><br/><br/>
    Password :<input type="password" name="pwd"></input><br/><br/>
    <input type="submit" value="Login"></input>
  </form>
</body>
</html>
```

Output:

When the user clicks on the "Login" button the following PHP script file gets executed. The PHP script reads the data from HTML form elements using \$_POST superglobal variable and then displays on the screen.

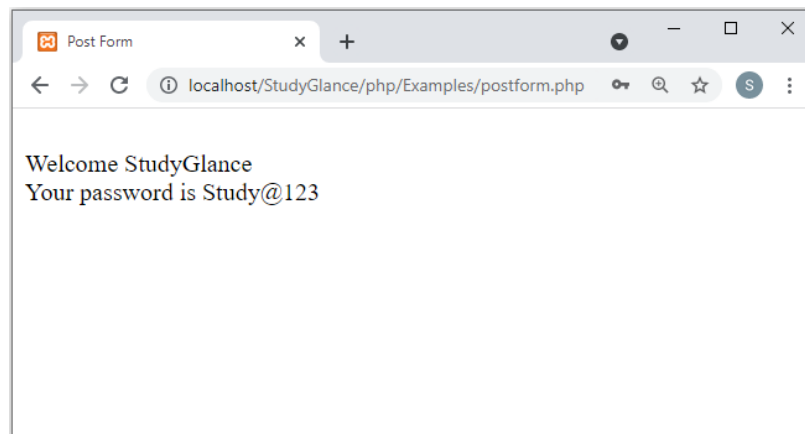
postform.php

```
<!doctype html>
<html lang="en">
  <title>Post Form</title>
</head>
```

```

<body><br>
<?php
    // To read data from input elements using $_POST
    $uname = $_POST["uname"];
    $password = $_POST["pwd"];
    // To display data
    echo "Welcome ".$uname."<br/>";
    echo "Your password is ".$password;
?>
</body>
</html>

```

Output:**Request Form**

PHP uses a special variable called `$_REQUEST` to collect data from forms.

It can save data sent using GET, POST or COOKIE methods. This parameter controls the order in which PHP looks for GET, POST and COOKIE data.

If you run a PHP script from the command line, the command line parameters will not be in the `$_REQUEST` array. However, the arguments are filled by the web server in the `$_SERVER` array.

GET Method (using `$_REQUEST`)

- When a form uses the GET method, form data is appended to the URL as query string (e.g., `?name=Divya&age=22`).
- PHP automatically stores this data in the `$_GET` superglobal array.
- The `$_REQUEST` array can also access the same data (since it combines GET, POST, and COOKIE).
- Data sent via GET is visible in the browser, can be bookmarked, and has a length limit.
- GET with `$_REQUEST` is best for non-sensitive data like search queries, filters, or navigation parameters.

HTML Form (get_form.html)

```

<!DOCTYPE html>
<html>
<head>
  <title>GET Form Example</title>
</head>
<body bgcolor="lightyellow">
  <h2>Form Handling with GET</h2>
  <form action="get_process.php" method="get">
    Name: <input type="text" name="name"><br>
    Age: <input type="number" name="age"><br>
    <input type="submit" value="Submit via GET">
  </form>
</body>
</html>

```

getprocess.php

```

<?php
// Handling GET request with $_REQUEST
if (isset($_REQUEST['name']) && isset($_REQUEST['age'])) {
  $name = $_REQUEST['name'];
  $age = $_REQUEST['age'];
  echo "Hello $name, you are $age years old. (Request via GET)";
}
?>

```

Output:

When you submit, the URL will look like:

http://localhost/get_process.php?name=Divya&age=22

Note: For post_request example ,replace the get with post

Reading data from Radio Button

RadioDemo.html

```
<html>
<head>
  <title>Radiobutton Example</title>
</head>
<body>
  <form action="radio.php" method="post">
    <b>1.PHP is a _____ language?</b>
    <br/><br/>
    <input type="radio" name="q1" value="Programming">&nbsp;Programming</input><br/>
    <input type="radio" name="q1" value="Server-side Scripting">&nbsp;Server-side
Scripting</input><br/>
    <input type="radio" name="q1" value="Client-side Scripting">&nbsp;Client-side
Scripting</input><br/>
    <input type="radio" name="q1" value="None of the above">&nbsp;None of the above</input><br/>
    <br/>
    <input type="submit" value="Show"/>
  </form>
</body>
</html>
```

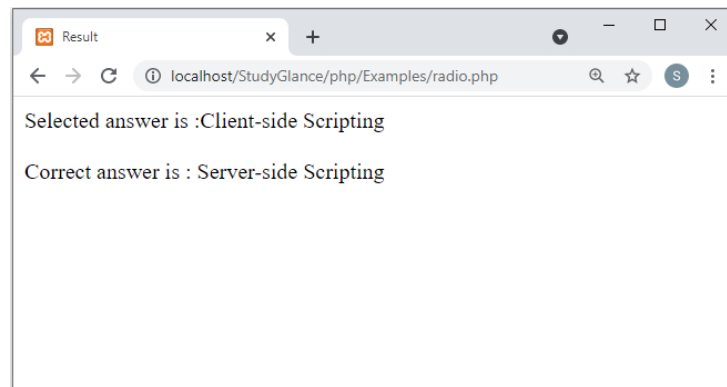
Output:

radio.php

```
<html>
<head>
  <title>Result</title>
</head>
```

```
<body>
<?php
    $ans=$_POST['q1'];
    echo "Selected answer is :$ans <br/><br/>";
    echo "Correct answer is : Server-side Scripting";
?>
</body>
</html>
```

Output:



Reading data from Selection List(Dropdown List)

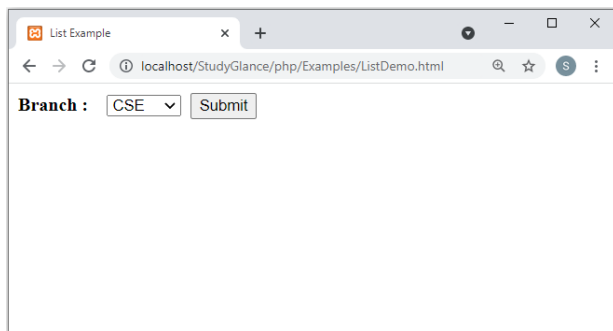
ListDemo.html

```
<html>  
  <head>  
    <title>List Example</title>  
  </head>  
  <body>  
    <form action="list.php" method="post">  
      <b>Branch : </b>&nbsp;&nbsp; <br>  
      <select name="branch">  
        <option value="CSE">CSE</option>  
        <option value="ECE">ECE</option>  
        <option value="EEE">EEE</option>  
        <option value="MECH">MECH</option>  
        <option value="CIVIL">CIVIL</option>  
      </select>  
      &nbsp;<input type="submit" value="Submit"/>  
    </form>
```

```
</body>
```

```
</html>
```

Output:



list.php

```
<html>
```

```
<head>
```

```
<title>List</title>
```

```
</head>
```

```
<body>
```

```
<?php
```

```
    $branch=$_POST['branch'];
```

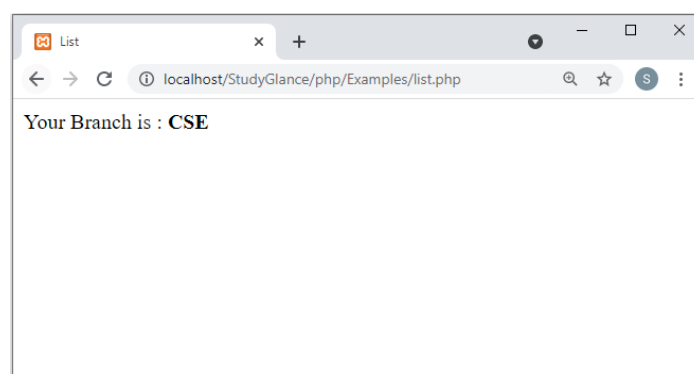
```
    echo "Your Branch is :&nbsp;  <b>$branch </b><br/><br/>";
```

```
?>
```

```
</body>
```

```
</html>
```

Output:



HANDLING FILE UPLOADS

- When a file is uploaded via an HTML form:
- The form must use the attribute `enctype="multipart/form-data"` to properly send file data.
- The file data is temporarily stored on the server in the `$_FILES` superglobal array.
- PHP provides the function `move_uploaded_file()` to move the uploaded file from temporary storage to a permanent location.
- The file size and type should be validated (using `$_FILES['file']['size']` and `$_FILES['file']['type']`) to ensure security and prevent malicious uploads.
- **Error codes** are provided in `$_FILES['file']['error']` to detect issues like file too large, incomplete upload, or no file selected.
- Uploaded files are first saved in a **temporary directory** defined by PHP (`php.ini` → `upload_tmp_dir`) until explicitly moved.
- The **target folder must have proper write permissions**, otherwise file saving will fail.
- For better security, files should be **renamed or stored outside the web root**, to avoid overwriting and direct execution.

Important Elements:

1. HTML Form with `enctype="multipart/form-data"`

To upload files, the form must use the attribute `enctype="multipart/form-data"`.

2. Input Field of Type File

The form must have `<input type="file" name="myfile">` to allow users to select a file from their computer.

3. HTTP POST Method

File uploads only work with the POST method, because GET cannot handle large binary data.

4. Temporary Storage of File

When a file is uploaded, PHP stores it in a temporary directory on the server. The location is given by `$_FILES['file']['tmp_name']`.

5. `$_FILES` Superglobal Array

PHP provides the `$_FILES` array to access file details:

`$_FILES['file']['name']` → Original file name

`$_FILES['file']['type']` → MIME type (e.g., image/jpeg)

`$_FILES['file']['size']` → Size in bytes

`$_FILES['file']['tmp_name']` → Temporary path

`$_FILES['file']['error']` → Error code (0 means no error)

6. `move_uploaded_file()` Function

Used to move the uploaded file from the temporary folder to a permanent location.

Syntax: `move_uploaded_file($_FILES['file']['tmp_name'], "uploads/filename");`

7. File Size Limit

Maximum file size can be controlled in php.ini using upload_max_filesize and post_max_size.

Developers should also check size manually using \$_FILES['file']['size'].

8. Allowed File Types

It's important to restrict uploads to safe types (e.g., images, pdfs).

This prevents users from uploading harmful files like .exe.

9. Upload Directory Permissions

The target folder (e.g., uploads/) must have proper write permissions for the server.

10. Error Handling

Always check \$_FILES['file']['error'] to handle issues like file too large, partial uploads, or no file selected.

11. Security Considerations

Never trust the file name or MIME type blindly; always validate and sanitize.

Store uploaded files outside the web root if possible.

Rename files to avoid overwriting and path traversal attacks.

Example: Upload an Image File

HTML Form (upload.html)

```
<!DOCTYPE html>

<html>
<head>
  <title>File Upload Example</title>
</head>
<body bgcolor="lightyellow">
  <h2>Upload a File</h2>
  <form action="upload.php" method="post" enctype="multipart/form-data">
    Select file: <input type="file" name="myfile"><br><br>
    <input type="submit" name="submit" value="Upload File">
  </form>
</body>
</html>
```

PHP Script (upload.php)

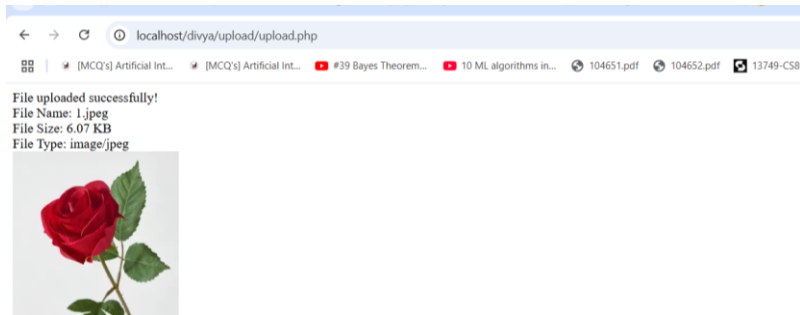
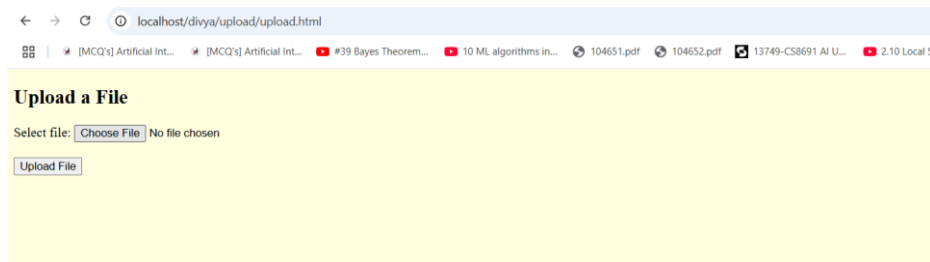
```
<?php
if (isset($_POST['submit'])) {
  // Folder where uploaded files will be saved
  $target_dir = "uploads/";
  // Get file details
  $file_name = $_FILES["myfile"]["name"];
```

```

$file_tmp = $_FILES["myfile"]["tmp_name"];
$file_size = $_FILES["myfile"]["size"];
$file_type = $_FILES["myfile"]["type"];
$file_error = $_FILES["myfile"]["error"];
// Create uploads folder if not exists
if (!is_dir($target_dir)) {
    mkdir($target_dir, 0777, true);
}
// Full path for saving file
$target_file = $target_dir . basename($file_name);
// Check for errors
if ($file_error === 0) {
    // Restrict file size (max 2MB)
    if ($file_size > 2 * 1024 * 1024) {
        echo "Error: File size is too large (max 2MB allowed).";
    } else {
        // Restrict file type (only images)
        $allowed_types = ["image/jpeg", "image/png", "image/gif"];
        if (in_array($file_type, $allowed_types)) {
            if (move_uploaded_file($file_tmp, $target_file)) {
                echo "File uploaded successfully!<br>";
                echo "File Name: " . $file_name . "<br>";
                echo "File Size: " . round($file_size / 1024, 2) . " KB<br>";
                echo "File Type: " . $file_type . "<br>";
                echo "<img src='$target_file' width='200'>";
            } else {
                echo "Error: Failed to move uploaded file.";
            }
        }
    }
} else {
    echo "Error: Only JPG, PNG, and GIF files are allowed.";
}
} else {
    echo "Error: File upload error code $file_error.";
}
}

```

?>

Output:**PHP MYSQL DATABASE INTRODUCTION**

[MySQL](#) is an open-source relational database management system (RDBMS). It is the most popular database system used with PHP. MySQL is developed, distributed, and supported by Oracle Corporation. The data in a MySQL database are stored in tables which consists of columns and rows. MySQL is a database system that runs on a server MySQL is ideal for both small and large applications. MySQL is very fast, reliable, and easy to use database system. It uses standard SQL.

There are three ways of working with MySQL and PHP

1. MySQLi (object-oriented)
2. MySQLi (procedural)
3. PDO(PHP data objects)

Database Syntaxes of Object oriented, Procedure oriented, PDO oriented**Step 1: Connection**

- **MySQLi (OOP):**

```
$conn = new mysqli($host, $user, $pass, $dbname);
```

- **MySQLi (Procedural):**

```
$conn = mysqli_connect($host, $user, $pass, $dbname);
```

- **PDO:**

```
$pdo = new PDO("mysql:host=$host;dbname=$dbname", $user, $pass);
```

Step 2: Create Database

- **MySQLi (OOP):**
\$conn->query("CREATE DATABASE dbname");
- **MySQLi (Procedural):**
mysqli_query(\$conn, "CREATE DATABASE dbname");
- **PDO:**
\$pdo->exec("CREATE DATABASE dbname");

Step 3: Create Table

- **MySQLi (OOP):**
\$conn->query("CREATE TABLE students (id INT PRIMARY KEY, name VARCHAR(50))");
- **MySQLi (Procedural):**
mysqli_query(\$conn, "CREATE TABLE students (id INT PRIMARY KEY, name VARCHAR(50))");
- **PDO:**
\$pdo->exec("CREATE TABLE students (id INT PRIMARY KEY, name VARCHAR(50))");

Step 4: Insert Data

- **MySQLi (OOP):**
\$conn->query("INSERT INTO students VALUES(1, 'Ravi')");
- **MySQLi (Procedural):**
mysqli_query(\$conn, "INSERT INTO students VALUES(1, 'Ravi')");
- **PDO:**
\$pdo->exec("INSERT INTO students VALUES(1, 'Ravi')");

Step 5: Update Data

- **MySQLi (OOP):**
\$conn->query("UPDATE students SET name='Divya' WHERE id=1");
- **MySQLi (Procedural):**
mysqli_query(\$conn, "UPDATE students SET name='Divya' WHERE id=1");
- **PDO:**
\$pdo->exec("UPDATE students SET name='Divya' WHERE id=1");

Step 6: Delete Data

- **MySQLi (OOP):**

```
$conn->query("DELETE FROM students WHERE id=1");
```

- **MySQLi (Procedural):**

```
mysqli_query($conn, "DELETE FROM students WHERE id=1");
```

- **PDO:**

```
$pdo->exec("DELETE FROM students WHERE id=1");
```

Step 7: WHERE Clause

- **MySQLi (OOP):**

```
$result = $conn->query("SELECT * FROM students WHERE age > 20");
```

- **MySQLi (Procedural):**

```
$result = mysqli_query($conn, "SELECT * FROM students WHERE age > 20");
```

- **PDO:**

```
$stmt = $pdo->query("SELECT * FROM students WHERE age > 20");
```

Step 8: Close Connection

- **MySQLi (OOP):**

```
$conn->close();
```

- **MySQLi (Procedural):**

```
mysqli_close($conn);
```

- **PDO:**

```
$pdo = null;
```

Comparison Table

Feature	MySQLi (OOP)	MySQLi (Procedural)	PDO (OOP)
Connection	<pre>\$conn = new mysqli(\$host, \$user, \$pass);</pre>	<pre>\$conn = mysqli_connect(\$host, \$user, \$pass);</pre>	<pre>\$pdo = new PDO("mysql:host=\$host;d bname=\$db", \$user, \$pass);</pre>
Create DB	<pre>\$conn-> query("CREATE DATABASE dbname");</pre>	<pre>mysqli_query(\$conn, "CREATE DATABASE dbname");</pre>	<pre>\$pdo->exec("CREATE DATABASE dbname");</pre>
Create Table	<pre>\$conn-> query("CREATE TABLE students (...");</pre>	<pre>mysqli_query(\$conn, "CREATE TABLE students (...)");</pre>	<pre>\$pdo->exec("CREATE TABLE students (...");</pre>

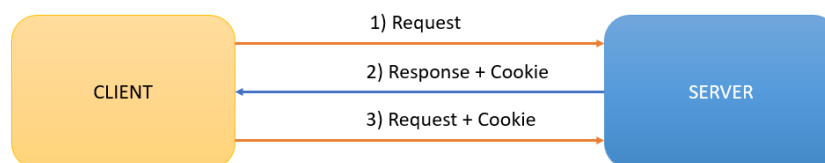
Insert Data	<code>\$conn->query("INSERT INTO students VALUES(...)");</code>	<code>mysqli_query(\$conn, "INSERT INTO students VALUES(...)");</code>	<code>\$pdo->exec("INSERT INTO students VALUES(...)");</code>
Update Data	<code>\$conn->query("UPDATE students SET name='...' WHERE id=1");</code>	<code>mysqli_query(\$conn, "UPDATE students SET name='...' WHERE id=1");</code>	<code>\$pdo->exec("UPDATE students SET name='...' WHERE id=1");</code>
Delete Data	<code>\$conn->query("DELETE FROM students WHERE id=1");</code>	<code>mysqli_query(\$conn, "DELETE FROM students WHERE id=1");</code>	<code>\$pdo->exec("DELETE FROM students WHERE id=1");</code>
WHERE	<code>\$result = \$conn->query("SELECT * FROM students WHERE age>20");</code>	<code>\$result = mysqli_query(\$conn, "SELECT * FROM students WHERE age>20");</code>	<code>\$stmt = \$pdo->query("SELECT * FROM students WHERE age>20");</code>
Close Connection	<code>\$conn->close();</code>	<code>mysqli_close(\$conn);</code>	<code>\$pdo = null;</code>

HANDLING COOKIES IN PHP

A cookie is a piece of information that is stored in the client [browser](#). Additionally, a cookie is generated on the server and saved to the client browser.

- A cookie is often used to identify a user.
- The Cookies are stored on the client side.
- A cookie takes less space to store data.

The first time when a client sends a request to the server, the server embedded the cookie along with the response. Next time onwards client sends the request along with a cookie to the server. In such a way, cookies can be received on the server side.



A cookie is a small file that the server embeds on the user's computer. With PHP, we can both create and retrieve cookie values.

Create cookies with PHP :

setcookie() function is used to set/create cookie.

Syntax:

```
setcookie(name, value, expire);
```

In the above only the **name** parameter is required. All other parameters are optional.

```
setcookie("username","Sreyas",time()+1*60*60);
```

In the above example, defining a name and value for cookie along with expiry time (1 hour = 1*60*60 seconds or 3600 seconds)

Retrieve a cookie :

\$_COOKIE superglobal variable is used to get/retrieve cookie. i.e Once cookie is set, we can access cookie by **\$_COOKIE** superglobal variable.

Example:

```
$value = $_COOKIE["username"];
```

HANDLING SESSIONS IN PHP

A session is a small piece of information which is stored at server side. And session is used to store and pass information from one page to another temporarily (until user close the website).

A session technique is widely used in shopping websites where we need to store and pass cart information e.g. user name, product code, product name, product price etc from one page to another.

In simple words, A session is a way to store information (in variables) to be used across multiple pages.

Start a PHP session :

session_start() function is used to start the session. It starts a new or resumes existing session.

Example:

```
session_start();
```

Note : The *session-start()* function must be the very first thing in your document.

To get and set session variables :

\$_SESSION is an associative array that contains all session variables. And it is used to set and get session variable values.

Example: To set session variable or store information in session variable

```
$_SESSION["name"] = "StudyGlance";
```

Example: To get session variable or get information from session variable

```
echo $_SESSION["name"];
```

login-style example using sessions. Let's build a **simple login system** with two PHP files:

File 1: login.php

```
<?php
```

```
// Start session
```

```
session_start();
```

```
// If form is submitted
```

```
if ($_SERVER["REQUEST_METHOD"] == "POST") {
```

```
    $username = $_POST['username'];
```

```
    $password = $_POST['password'];
```

```
// Simple login check (You can replace with database check)
if ($username == "admin" && $password == "1234") {
    $_SESSION["username"] = $username; // Store in session
    header("Location: welcome.php"); // Redirect to welcome page
    exit();
} else {
    echo "Invalid username or password!";
}
}
?>

<!DOCTYPE html>
<html>
<head>
    <title>Login Page</title>
</head>
<body>
    <h2>Login Form</h2>
    <form method="post" action="">
        Username: <input type="text" name="username" required><br><br>
        Password: <input type="password" name="password" required><br><br>
        <input type="submit" value="Login">
    </form>
</body>
</html>
```

File 2: welcome.php

```
<?php
// Start session
session_start();

// Check if user is logged in
if (!isset($_SESSION["username"])) {
    echo "You are not logged in! <a href='login.php'>Login here</a>";
    exit();
}

// Display session variable
echo "<h2>Welcome, " . $_SESSION["username"] . "!"</h2>";
echo "<a href='logout.php'>Logout</a>";
?>
```

File 3: logout.php

```
<?php
// Start session
session_start();

// Destroy session
session_destroy();
```

```
echo "You have been logged out! <a href='login.php'>Login again</a>";
?>
```

How It Works

1. Open login.php → Enter **username: admin, password: 1234**.
2. If correct → Redirects to welcome.php and shows:
3. Welcome, admin!
4. [Logout]
5. If wrong → Shows **Invalid username or password!**
6. Click logout → logout.php clears session and asks to login again.

FILE HANDLING IN PHP

In PHP File System allows us to create file, read file line by line, read file character by character, write file, append file, delete file and close file.

PHP supports following functions to handle files.

- **fopen()**
- **fread()**
- **fwrite()**
- **fappend()**
- **fclose()**
- **unlink()**

(1)fopen()

In PHP, The fopen() function is used to open a file in read mode or write mode or in both read and write modes. And this function accepts two arguments: **\$filename** and **\$mode**.

Syntax: fopen(string \$filename , string \$mode)

The **\$filename** represents the file to be opened and **\$mode** represents the file mode for example read-only, read-write, write-only etc.

The **fopen()** supports the following modes:

- R - Read Only
- W - Write Only
- A - Append
- r+ - Read & Write.
- w+ - Read & Write.

Example: "fopen.php"

```
<?php
```

```
$filename = "E:\\studyglance\\simple.txt";
```

```
$fp = fopen($filename, "r");//open file in read mode
```

?>

(2)fread()

In PHP, The fread() function is used to read data of the file. It requires two arguments: **file resource** and **file size**.

Syntax: fread(resource \$fp,int \$length)

Where **\$fp** represents file pointer that is created by fopen() function. And **\$length** represents length of file (bytes)to be read.

Example: "fread.php"

<?php

```
$filename = "D:\\studyglance\\file.txt";
```

```
$fp = fopen($filename, "r");//open file in read mode
```

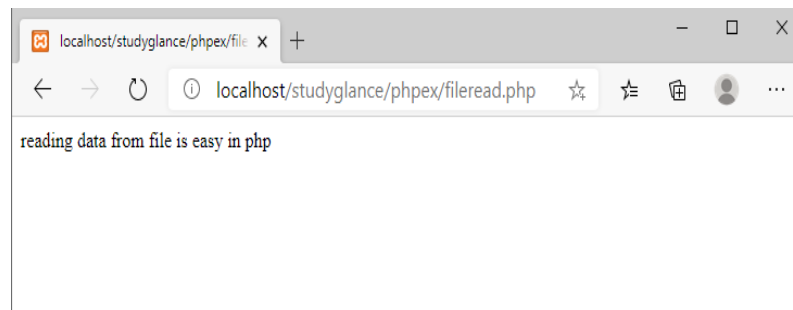
```
$content = fread($fp, filesize($filename));//read file
```

```
echo "$content";//printing data of file
```

```
fclose($fp);//close file
```

?>

Output :



(3) fwrite()

In PHP, The fwrite() function is used to write the content (string) into file. And It requires two arguments: **file resource** and **data(string)**

If the file opened in write(**w**) mode, the fwrite() function will erase the previous data of the file and writes the new data or If the file opened in append(**a**) mode, the fwrite() function appends the new data at the end of the previous data.

Syntax: fwrite(resource \$fp,string \$data)

Where **\$fp** represents file pointer that is created by fopen() function. And **\$data** represents the data to be written.

If the file opened in **write(w) mode :**

Example: "fwrite.php"

```
<?php
```

```
$filename = "D:\\studyglance\\file.txt";
```

```
$fp = fopen($filename, "w");//open file in write mode
```

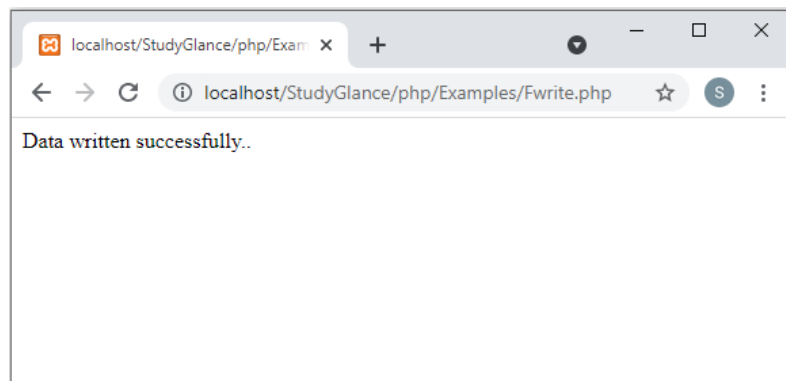
```
fwrite($fp,'PHP is a Scripting Language');
```

```
echo "Data written successfully..";
```

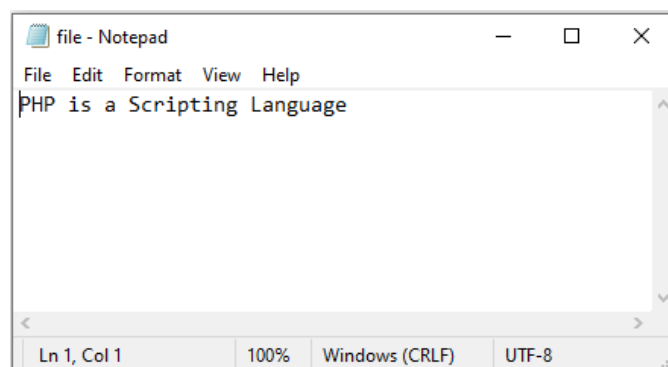
```
fclose($fp);//close file
```

```
?>
```

Output :



Text File :



(4) fappend()

If the file opened in **append(a) mode** :

Example: "fappend.php"

```
<?php
```

```
$filename = "D:\\studyglance\\file.txt";
```

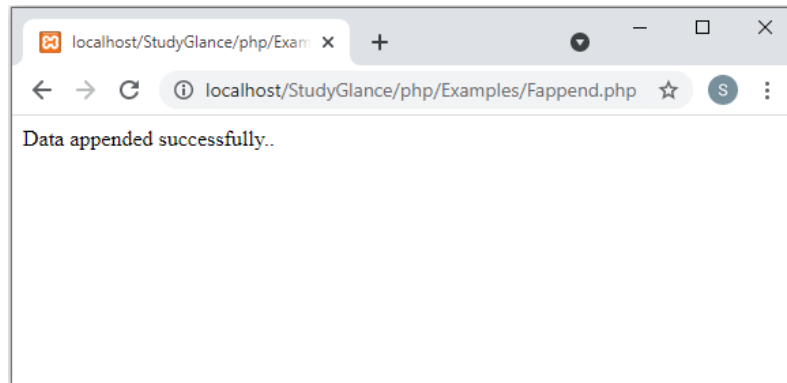
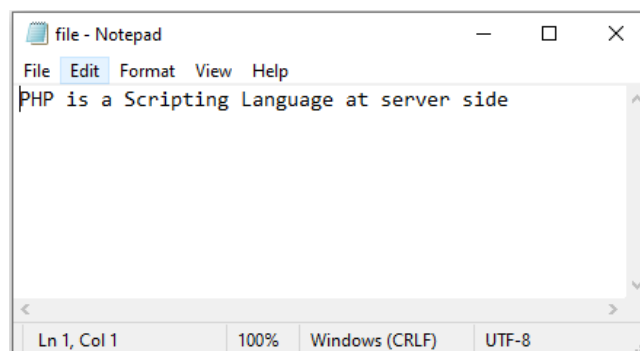
```
$fp = fopen($filename, "a");//open file in write mode
```

```
fwrite($fp,' at server side');
```

```
echo "Data appended successfully..";
```

```
fclose($fp);//close file
```

```
?>
```

Output :**Text File :****(5) fclose()**

In PHP, The fclose() function is used to close an open file. And It requires one argument i.e. "**File Pointer**"

Syntax: fclose(resource \$fp)

Where **\$fp** represents file pointer that is created by fopen() function.

Example: "fclose.php"

```
<?php
```

```
$filename = "D:\\studyglance\\file.txt";
```

```
$fp = fopen($filename, "r");//open file in write mode
```

```
// To close the file
```

```
fclose($fp);
```

```
?>
```

(6) unlink()

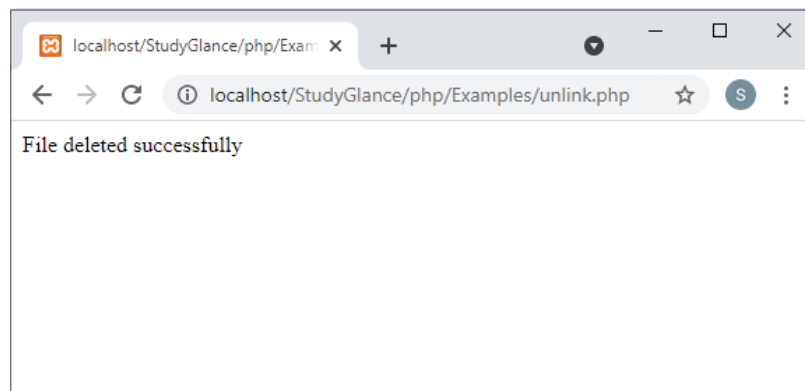
In PHP, The unlink() function is used to delete any file. And It requires one argument i.e. "**File Name**"

Syntax: unlink(string \$filename)

Where **\$filename** represents name of the file to be remove.

Example: "unlink.php"

```
<?php
$status=unlink('file.txt');
if($status){
    echo "File deleted successfully";
}else{
    echo "Sorry!";
}
?>
```

Output :**PHP - DIRECTORY FUNCTIONS**

- In the computer system folders are referred to as a directories
- PHP directory functions are a group of functions that can be used to get, modify, and retrieve information about various file system directories and their specific contents.
- Directories can be used for multiple types of tasks, like adding, removing, changing the current working directory, showing the files include in the directory, and more.

PHP Directory Functions

Function	Description
chdir()	Changes the current directory
chroot()	Changes the root directory
closedir()	Closes a directory handle
dir()	Returns an instance of the Directory class
getcwd()	Returns the current working directory

<u>opendir()</u>	Opens a directory handle
<u>readdir()</u>	Returns an entry from a directory handle
<u>rewinddir()</u>	Resets a directory handle
<u>scandir()</u>	Returns an array of files and directories of a specified directory